

Ver el error de hoy y acordar qué construimos

Un agente que mapea el catálogo a Shopee, decidido paso a paso.

War Room · Alephee × Craftech × AWS · 1/10/2026

Ver el error de hoy y acordar qué construimos

Lo que vamos a hacer hoy.

Decidimos → construimos → medimos. Cada bloque cierra con una versión que funciona.

- 1 · PUNTO DE PARTIDA | El error de hoy, alcance y contrato
- 2 · DISEÑAR EL AGENTE | Tipo de aplicación, prompt, dónde corre, modelo, stack y criterio de éxito
- 3 · V1 · EL AGENTE RESPONDE | Solo instrucciones: qué falla y por qué
- 4 · V2 · HERRAMIENTAS | Las tablas como herramientas, el loop y la caché de prompt
- 5 · V3 · CONTROL | Guardrails, memoria y caché por SKU
- 6 · LA PRUEBA | Hoy contra V1, V2 y V3, y el camino a producción

Ver el error de hoy y acordar qué construimos

De un producto de Alephee a una publicación de Shopee.

Caso real · SKU 88904447 · Correia dentada · catálogo GM Brasil (Mercado Libre Brasil) → Shopee Brasil

Entra (Alephee)	Sale (Shopee)	Quién lo resuelve
Categoría: Correias Dentadas	Distribuição e Correias	La tabla
Condición del ítem: Novo	Condition: New	La tabla y la lista del canal
Tiempo de garantía: 6 meses	Duração da Garantia: 6 Months	El agente
Código OEM: no viene	Faltante, con motivo (hoy: ABS Plastic, rechazado por Shopee)	El guardrail

Ver el error de hoy y acordar qué construimos

Hoy: dos llamadas, un merge y un presupuesto que se agota.

- Categoría: la tabla `reference_category` y, si no alcanza, el modelo elige
- Atributos: dos llamadas en paralelo (con y sin referencia) y un merge por código
- USD 350 por mes: cuando se agota, se publica sin atributos

Ver el error de hoy y acordar qué construimos

Lo que Shopee rechazó y lo que aceptó mal.

Datos reales · WarRoom.zip del 24/09 · 30 productos

Caso	Qué salió	Qué pasó
88904447	Código OEM = ABS Plastic	Rechazado: el valor no pertenece al atributo
93221445	Inmetro Certification con varios valores	Rechazado: solo admite un valor
9 de 30	Valor -1 publicado como valor	Aceptado por Shopee
98550368	14 de 15 URN sin sufijo del canal	Aceptado por Shopee

Ver el error de hoy y acordar qué construimos

¿Qué hace y qué no hace?

DECISIÓN 1

¿Qué alcance tiene el agente del war room?

A · Un canal (Shopee) y una familia de productos: categoría + atributos

B · Todos los canales conectados (Shopee, Magalu, Tienda Nube...)

C · También título, descripción y marca

PROPUESTA · A. Un canal y una familia alcanzan para decidir el método y medirlo. Los prompts de marca, título y descripción son generación libre, no mapeo: quedan fuera. Los otros canales replican el mismo diseño con sus propias tablas.

decisiones/01-alcance.md

Ver el error de hoy y acordar qué construimos

Entra un producto. Sale una propuesta de publicación.

- Entrada: el producto de Alephee (SKU, nombre, descripción, categoría legacy, atributos)
- Salida: la categoría de Shopee más los atributos con URN, valueId y valor
- Y dos listas más: missing (obligatorios sin dato) y rejected (atributos descartados, con motivo)

Ver el error de hoy y acordar qué construimos

¿Qué recibe y qué entrega?

DECISIÓN 2

¿Cómo se informa lo que no se pudo mapear?

A · No se informa: se publica lo que hay (hoy)

B · missing y rejected con motivo, en la misma salida

C · Un archivo de log aparte que alguien revisa

PROPUESTA · B. El faltante viaja con la publicación: quien revisa ve el producto, el atributo y el motivo juntos. Un log aparte se separa del dato y nadie lo mira. La decisión 12 define qué pasa después con esas listas.

decisiones/02-contrato.md

Tomar las decisiones que definen la V1

Diseñar antes de escribir.

Siete decisiones que definen la V1

Tomar las decisiones que definen la V1

Cuatro formas de usar un modelo.

- SINGLE PROMPT | Una llamada, una respuesta. Barato y fácil de evaluar; no consulta nada
- WORKFLOW | Pasos fijos en código; el modelo participa en uno o dos. Predecible
- AGENTE | El modelo decide qué herramienta pedir y cuándo entregar, dentro de un límite de rondas
- MULTIAGENTE | Varios agentes con roles distintos. Solo cuando uno no alcanza

Tomar las decisiones que definen la V1

¿Single prompt o agente?

La opción A ya está en la base preparada (V1) · si gana otra, se discute qué cambia

DECISIÓN 3

¿Con qué tipo de aplicación empezamos?

A · Single prompt (V1) y agregar herramientas solo cuando un fallo lo justifique

B · Agente con herramientas desde el inicio

C · Workflow determinista sin modelo

PROPUESTA · A. La V1 muestra qué resuelve el modelo solo y qué no; cada fallo justifica la capa siguiente y queda documentado. C no alcanza: elegir el valor de lista equivalente (por ejemplo, '1' → Sim) necesita interpretación.

decisiones/03-tipo-de-aplicacion.md

Tomar las decisiones que definen la V1

Un prompt tiene cinco partes.

01 Rol: quién es

02 Tarea: qué hace y con qué entrada

03 Reglas: qué nunca hace

04 Formato: cómo entrega

05 Cuando falta dato: qué hace

Tomar las decisiones que definen la V1

Buenas prácticas que cambian el resultado.

- Lo estático primero y lo variable al final: habilita la caché de prompt (F4, F7)
- Salida estructurada por esquema, no JSON pedido en el texto
- Decir qué hacer cuando no sabe, no solo qué hacer

Tomar las decisiones que definen la V1

¿Cómo garantizamos el formato?

La opción B ya está en la base preparada · si gana otra, se discute qué cambia

DECISIÓN 4

¿Cómo se garantiza que la salida cumpla el contrato?

A · Pedir JSON en el texto y parsear (hoy)

B · Herramienta con esquema Pydantic: la entrega es una llamada tipada

C · Post-procesar la respuesta con expresiones regulares

PROPUESTA · B. El formato deja de ser una regla del prompt y pasa a ser un contrato que el modelo no puede violar. C arregla síntomas: si el modelo omite un campo, la regex no lo inventa.

decisiones/04-salida-estructurada.md

Tomar las decisiones que definen la V1

Dónde corre: AgentCore o contenedor propio.

Fuentes F1–F3b en docs/warroom/fuentes.md

- AGENTCORE RESUELVE | Runtime sin servidores, identidad, memoria, gateway de herramientas y observabilidad, como servicios separados que se usan juntos o no (F2)
- DUDAS | Arranque en frío sin cifra publicada, sesiones que terminan a los 15 min de inactividad y 8 h máximo (F3b), costo por CPU y memoria de la sesión (F3), región disponible
- ALTERNATIVA | Contenedor o Lambda propios: más control y más trabajo de operación; el agente es el mismo

Tomar las decisiones que definen la V1

La infraestructura, de punta a punta.

Tomar las decisiones que definen la V1

¿Dónde corre?

DECISIÓN 5

¿Dónde corre el agente?

A · Hoy en local; el chat de demo en AgentCore; el batch como proceso de Alephee, a decidir con el camino a producción

B · Todo en AgentCore desde el inicio

C · Todo en la infraestructura actual de Alephee

PROPUESTA · A. Hoy se construye y se mide en local. El chat va a AgentCore porque es su caso natural. Para el batch hace falta medir sesiones, costo y arranque antes de decidir; se retoma a las 16:15.

decisiones/05-donde-corre.md

Tomar las decisiones que definen la V1

Elegir el modelo: qué pesa en este caso.

- Seguir reglas y usar herramientas sin inventar: es lo que más falla hoy
- Costo por token con caché de prompt: el presupuesto actual es de USD 350 por mes
- Latencia: no es restricción (hoy el batch tarda 13 s por producto)
- Disponible en la región y habilitado en la cuenta

Tomar las decisiones que definen la V1

Qué hay en Bedrock.

Fuente F6 · Models at a glance · verificado el 30/09/2026

Proveedor	Familias	Para este caso
Anthropic	Claude 5.x (Sonnet 5.5, Opus 5.5, Fable 5.1, Sonnet 5...), 4.x (Haiku 4.5...), 3.x	Candidato principal: reglas y tool calling. Sonnet 5 elegido el 25/09; 5.5 y Haiku 4.5 a comparar
Amazon	Nova 2 Lite, Nova Premier, Pro, Lite, Micro	Alternativa de costo a medir
OpenAI	GPT-5.x, GPT-6, GPT OSS	Continuidad con el proveedor actual, dentro de AWS
Meta · Mistral · DeepSeek · Qwen · otros	Llama 3.x y 4, Mistral Large 3, DeepSeek V3.2, Qwen3...	Abiertos; evaluar si el costo lo justifica

Tomar las decisiones que definen la V1

¿Qué modelo?

Tomada el 25/09 (CLAUDE.md, decisión 6) · hoy se valida o se cambia

DECISIÓN 6

¿Con qué modelo construimos y medimos?

A · Claude Sonnet 5 en Bedrock, vía Converse

B · Un modelo más chico y barato (Claude Haiku 4.5) y medir

C · Seguir con OpenAI y solo reordenar el prompt para la caché

PROPUESTA · A para construir hoy: es la decisión del 25/09 y el ID está verificado en Converse. Claude Sonnet 5.5 también está en Bedrock (F6): compararlo, junto con Haiku 4.5 (B), es una corrida más con otra variable de entorno y queda como pendiente con dueño. C mejora el costo pero no resuelve herramientas, contrato ni control.

decisiones/06-modelo.md

Tomar las decisiones que definen la V1

Stack y harness.

- Python + LlamaIndex Workflows: pasos, eventos y herramientas ya resueltos
- BedrockConverse: tool calling y caché de prompt sin código propio
- Harness de desarrollo: propone cambios en el código; Gastón los revisa y corre los tests

Tomar las decisiones que definen la V1

¿Con qué lo construimos?

Tomada el 28/09 (CLAUDE.md, decisión 9) · hoy se valida

DECISIÓN 7

¿Qué stack usamos para el agente?

A · Python + LlamaIndex Workflows + BedrockConverse

B · Llamadas directas al SDK, sin framework (como hoy)

C · Otro framework de agentes (Strands, LangGraph...)

PROPUESTA · A. Es el stack del template que Crafttech ya opera: el workflow no conoce Bedrock y cada paso se prueba con dobles. B repite el código de loop, herramientas y caché que un framework ya trae. C es válido; se elige A por continuidad con lo que ya está probado.

decisiones/07-stack.md

Tomar las decisiones que definen la V1

¿Cuándo está bien hecho?

Esquema de Shopee y salida esperada MOCK · validar con el equipo de catálogo

- Exacto: categoría correcta, ni sobra ni falta atributo, nada fuera de dominio, sin duplicados y faltantes informados
- Además se reportan precisión y recall de atributos, valores inválidos, tokens y segundos
- Dataset: 30 productos reales con su publicación actual; el esquema de atributos de Shopee y el expected son simulados

Tomar las decisiones que definen la V1

¿Cuál es el número que aceptamos?

DECISIÓN 8

¿Qué criterio de éxito acordamos para la prueba de las 16:15?

A · Cero valores inválidos, cero duplicados y todo obligatorio informado en los 30; los exactos se reportan pero no son condición hasta validar el expected con catálogo

B · Exactos iguales o mayores al 80 %

C · Solo precisión y recall de atributos

PROPUESTA · A. Con expected mock, 'exacto' castiga aciertos que hoy nadie mapea: el 28/09 el proceso actual dio 7 y las versiones 5, 4 y 4. Los inválidos, duplicados y obligatorios sin informar sí son errores seguros y hoy hay 47, 5 y 2. El número final se fija en la sala.

decisiones/08-criterio-de-exito.md

Tomar las decisiones que definen la V1

¿Con qué dataset?

DECISIÓN 9

¿Sobre qué datos construimos y medimos?

A · Los 30 reales del zip, con esquema y expected mock rotulados

B · Solo los 10 casos mock de borde

C · Esperar el esquema oficial de Shopee para empezar

PROPUESTA · A. Son productos reales con publicaciones reales y rechazos reales de Shopee. Lo simulado queda rotulado y se valida con catálogo después. Los 10 mock quedan como tests de borde.

decisiones/09-dataset.md

Construir, correr y leer la primera versión

V1 · el agente responde.

Solo instrucciones. Se espera que falle, y esos fallos justifican lo que sigue.

Construir, correr y leer la primera versión

Demo · un caso real por V1.

```
scripts/correr.sh --version v1 --datos real --caso error-88904447
```

- La categoría elegida frente a la de `reference_category`
- Cada atributo: ¿de dónde salió el valor? ¿Está en la lista del canal?
- `missing` y `rejected`: ¿informa lo que no pudo o completó igual?
- Tokens de entrada: todo el contexto del canal viaja en cada llamada

Respaldo · resultados/v1-real-20260928-173758.json (corrida del 28/09, anterior a las correcciones)

Construir, correr y leer la primera versión

Qué falló en V1 y qué capa lo resuelve.

Corrida del 28/09 · 30 reales (fila 1: caso MOCK 09) · anterior a las correcciones · expected MOCK

Fallo	Dónde se vio	Capa que lo resuelve
Categoría inventada cuando no hay referencia	Caso mock 09 (sin categoría de origen)	V2 · la tabla por herramienta
Valor fuera de la lista del canal	19 valores en los 30 reales	V3 · guardrail
Obligatorio sin informar en missing	2 casos en los 30 reales	V3 · guardrail
~22.000 tokens de entrada por producto	Todos los casos	V2 · herramientas + caché de prompt

Decidir qué resuelve la tabla y conectarla

V2 · herramientas.

El agente consulta antes de decidir.

Decidir qué resuelve la tabla y conectarla

Una herramienta es una función que el modelo pide y el código ejecuta.

01 El modelo pide `buscar_categoria('urn:category:1106872')`

02 El programa ejecuta la función: consulta la tabla

03 Devuelve `{encontrada: true, urn, name}` o `{encontrada: false, motivo}`

04 El modelo sigue con ese dato, no con su memoria

Decidir qué resuelve la tabla y conectarla

¿Qué decide la tabla y qué decide el agente?

En la base preparada: la categoría se fija en código (V3); los campos de la tabla los aplica el modelo por instrucción

DECISIÓN 10

¿Qué decide la tabla y qué decide el agente?

- A · La tabla fija la categoría y los campos; el agente solo elige el valor equivalente de la lista y cubre lo que la tabla no tiene
- B · El agente puede corregir la tabla si cree que está mal
- C · Todo por tabla; sin modelo

PROPUESTA · A. La tabla manda (acuerdo del 25/08 con Juan David). Hoy la categoría se impone en código (V3) y la tabla de atributos la consulta el agente por herramienta, desambiguada por categoría en código; imponer también los campos por código es el siguiente paso. Lo que la tabla no cubre (valores de lista, atributos sin referencia) es lo único que decide el agente.

decisiones/10-tabla-vs-agente.md

Decidir qué resuelve la tabla y conectarla

Costo por producto, medido.

Corrida del 28/09 · 30 reales · anterior a las correcciones · tokens de entrada por producto

Versión	Entrada sin caché	Leída de caché	Segundos
V1	~22.000	—	12,3
V2	~1.600	~12.000	18,3

Decidir qué resuelve la tabla y conectarla

¿Cuánto puede costar?

Abierta: hoy no hay tope ni medición completa en la base preparada

DECISIÓN 11

¿Cómo tratamos la restricción de costo del modelo?

A · Tope mensual acordado con Alephee y medición por producto: tokens de entrada, salida, caché leída y escrita

B · Sin tope: se mide después

C · El tope de hoy (USD 350) y publicar sin atributos al agotarse

PROPUESTA · A. Hoy el tope corta la calidad (C). Con caché de prompt y caché por SKU (decisión 13) el costo por producto baja; el número se fija con una corrida completa medida, no con esta tabla.

decisiones/11-costo.md

Decidir qué resuelve la tabla y conectarla

Demo · el mismo caso por V2.

```
scripts/correr.sh --version v2 --datos real --caso error-88904447
```

- Qué herramientas pidió y en qué orden (herramientas_usadas)
- La categoría ahora sale de la tabla, no del modelo
- Código OEM: ¿sigue siendo ABS Plastic?
- Tokens leídos de caché frente a los nuevos

Respaldo · resultados/v2-real-20260928-181509.json (corrida del 28/09, anterior a las correcciones)

Decidir qué pasa cuando el agente no sabe

V3 · control.

Qué pasa cuando el agente no sabe.

Decidir qué pasa cuando el agente no sabe

El agente por dentro: código y modelo.

Decidir qué pasa cuando el agente no sabe

Guardrail: una comprobación en código, no otra instrucción.

- REVISAR | Lista los problemas en lenguaje claro y se los devuelve al agente, que tiene una ronda para corregir
- LIMPIAR | Red final: descarta lo inválido con motivo y marca los obligatorios que faltan
- NUNCA | Publica un valor fuera de la lista del canal, un -1 ni un duplicado. No verifica que el valor sea verdad: eso sigue siendo del esquema y de catálogo

Decidir qué pasa cuando el agente no sabe

¿Qué hace cuando no sabe?

La opción B ya está en la base preparada (V3) · C se construye sobre B

DECISIÓN 12

¿Qué hace el agente cuando no puede completar un atributo obligatorio?

A · Publicar sin el atributo (hoy)

B · Faltante explícito con motivo: la publicación sale con missing y alguien decide

C · Bloquear la publicación hasta revisión humana

PROPUESTA · B. Reemplaza el publicar sin atributos. Quién recibe missing y si bloquea o no la publicación es la integración con la plataforma de Alephee: pendiente con dueño. C es una política válida que se puede construir sobre B.

decisiones/12-cuando-no-sabe.md

Decidir qué pasa cuando el agente no sabe

Memoria: correcciones del equipo de catálogo.

- Una corrección dice: en esta categoría, este valor del producto va a este atributo con este valor del canal
- El agente la consulta como una herramienta más y manda sobre su criterio
- Hoy es un archivo JSON local; en producción, AgentCore Memory o una tabla de Alephee

Decidir qué pasa cuando el agente no sabe

¿Cómo garantizamos determinismo?

La opción A ya está en la base preparada (clave SKU + categoría legacy) · falta la invalidación por tablas

DECISIÓN 13

¿Cómo garantizamos que el mismo SKU dé la misma salida?

A · Caché por SKU + canal; se invalida al corregir o al cambiar las tablas

B · Seed fijo (lo que se intentó hoy; los parámetros exactos siguen sin confirmar)

C · Recalcular siempre y aceptar variación

PROPUESTA · A. Un mapeo por SKU y canal. B reduce la variación pero no la elimina y sigue pagando cada corrida. La clave actual es SKU + categoría legacy; falta sumar versión de las tablas y aislamiento por cuenta antes de producción.

decisiones/13-determinismo-y-cache.md

Decidir qué pasa cuando el agente no sabe

Demo · corregir y repetir.

```
scripts/corregir.sh --categoria <urn-categoria> --urn <urn-atributo> --valor-producto <valor>  
--value-id <id> --value <nombre> scripts/correr.sh --version v3 --datos real --caso error-88904447  
--cache
```

- buscar_correcciones aparece en herramientas_usadas
- El valor corregido sale tal cual lo cargó catálogo
- rejected explica cada descarte del guardrail
- Segunda corrida del mismo caso: herramientas_usadas = [cache] y cero tokens

Respaldo · resultados/v3-real-20260928-180524.json (corrida del 28/09, anterior a las correcciones)

Medir contra el criterio y repartir lo que sigue

La prueba.

Hoy contra V1, V2 y V3, sobre los 30, con el criterio de la decisión 8.

Medir contra el criterio y repartir lo que sigue

Resultados (se completan en vivo).

Columna Hoy: publicaciones exportadas, expected MOCK. Las demás se llenan con la corrida del día

Métrica	Hoy	V1	V2	V3
Casos exactos	7	.	.	.
Categoría correcta	28	.	.	.
Valores inválidos (-1 o fuera de lista)	47	.	.	.
Duplicados	5	.	.	.
Obligatorios sin informar	2	.	.	.

Medir contra el criterio y repartir lo que sigue

Cómo leer la tabla.

- Exactos bajos en todas las columnas: el expected hereda las omisiones del proceso actual y castiga aciertos que hoy nadie mapea
- Inválidos y duplicados sí son errores seguros: la V3 no entrega ninguno detectable por este evaluador (28/09)
- Categoría: la V3 dio 28/30 en la corrida vieja; la corrección del 28/09 la fija desde la tabla y hay que volver a medir

Medir contra el criterio y repartir lo que sigue

Camino a producción.

01 Esquema oficial de Shopee y expected validado con el equipo de catálogo

02 Integración: la API pública lee el producto (F8); escribir publicaciones y leer tablas necesita acceso interno

03 Decidir dónde corre el batch (decisión 5) y la política de missing (decisión 12)

04 Medir el costo completo y fijar el tope (decisión 11); probar Haiku 4.5 (decisión 6)

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (1 a 7).

N	Decisión	Archivo
1	Alcance: un canal, una familia, categoría + atributos	decisiones/01-alcance.md
2	Contrato: missing y rejected con motivo	decisiones/02-contrato.md
3	Tipo de aplicación: single prompt y escalar	decisiones/03-tipo-de-aplicacion.md
4	Salida estructurada por herramienta + Pydantic	decisiones/04-salida-estructurada.md
5	Dónde corre: local hoy, chat en AgentCore, batch a medir	decisiones/05-donde-corre.md
6	Modelo: Claude Sonnet 5; Haiku 4.5 a probar	decisiones/06-modelo.md
7	Stack: Python + LlamaIndex Workflows + BedrockConverse	decisiones/07-stack.md

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (8 a 13).

N	Decisión	Archivo
8	Criterio de éxito: el número de la pizarra	decisiones/08-criterio-de-exito.md
9	Dataset: 30 reales con mock rotulado	decisiones/09-dataset.md
10	La tabla manda; el agente elige valores y lo no cubierto	decisiones/10-tabla-vs-agente.md
11	Costo: tope acordado y medición por producto	decisiones/11-costo.md
12	Cuando no sabe: faltante explícito, nunca inventar	decisiones/12-cuando-no-sabe.md
13	Determinismo: caché por SKU + canal con invalidación	decisiones/13-determinismo-y-cache.md

Medir contra el criterio y repartir lo que sigue

Quién hace qué, para cuándo.

Se completa en la sala.